IntelQcopter: Mathematical Model, Six Control Loops, State Estimation and Navigation
Detailed derivations and reproducible simulation evidence

IntelQcopter project

20 October 2026

Contents

1 Scope, evidence and physical assumptions **1**
- 1.1 What this document establishes . . . 1
- 1.2 Evidence boundaries . . . 2

2 Linear algebra from definitions **3**
- 2.1 Products, dimensions and ordering . . . 3
- 2.2 Covariance and its transformation . . . 3
- 2.3 Matrix powers and exponentials . . . 4
- 2.4 Two distinct process-noise conventions . . . 4

3 Coordinate frames and rigid-body mechanics **6**
- 3.1 Frames, rotations and signs . . . 6
- 3.2 Newton and Euler equations . . . 7
- 3.3 Motor geometry, allocation and saturation . . . 7
- 3.4 Numerical propagation . . . 8

4 Exactly six feedback controllers **9**
- 4.1 Signals and units . . . 9
- 4.2 Roll and pitch rate PI derivation . . . 9
- 4.3 Outer angle loops and yaw . . . 10
- 4.4 One altitude PID with vertical-speed stick command . . . 10
- 4.5 A reproducible gain family, not hidden hand tuning . . . 11

5 Kalman filtering: proof and implemented equations **12**
- 5.1 Derivation of the gain . . . 12
- 5.2 Scalar one-angle filter . . . 12
- 5.3 Actual MPU filter: one axis, two states . . . 13
- 5.4 Accelerometer observations and their limitation . . . 13
- 5.5 Altitude filter: state, measurements and expanded products . . . 14
- 5.6 Joseph multiplication in full detail . . . 14
- 5.7 Gravity compensation, lidar and observability . . . 15

6 Uncertainty, statistical checks and error accounting **16**
- 6.1 What standard deviation does and does not mean . . . 16
- 6.2 Innovation consistency and physical-parameter errors . . . 16
- 6.3 Separate error sources . . . 17

7 Companion navigation and perception **18**
- 7.1 A bounded static-map baseline . . . 18
- 7.2 ROS 2 data contract . . . 18
- 7.3 Why YOLO is not a range sensor . . . 19

8 Reproduction and next physical measurements **20**

9 Recorded numerical results **21**
9.1 Six PID gain tables . . . 21
9.1.1 Baseline . . . 21
9.1.2 Selected simulation . . . 21
9.2 Selection and route suite . . . 21
9.3 Kalman gains, standard deviation and distributions . . . 24
9.3.1 Independent Monte Carlo experiments . . . 24
9.4 Numerical refinement and algebra checks . . . 26
9.5 Executed ROS 2 communication test . . . 26
9.6 Executed YOLO and USD tests . . . 26
9.7 Files and reproducibility . . . 27

Chapter 1

Scope, evidence and physical assumptions

1.1 What this document establishes

This report derives an explicit six-degree-of-freedom model, a six-controller architecture, the covariance and gain calculations used by the C estimators, and a bounded local-navigation baseline. Numerical results are generated by the production C control equations and a common C++ navigation core. They establish behaviour under stated simulation assumptions. They do not identify the actual aircraft or certify a flight tune.

The physical aircraft is an F450/Q450-style X quadcopter with 9047 propellers, nominal 1400 KV motors, four 40 A ESCs and a 3S 4200 mAh LiPo. The approximately 150 g battery mass is unverified. A motor's KV value and an ESC's current rating do not determine thrust, propeller efficiency or inertia. In particular, 40 A is not a thrust measurement. The following complete-vehicle model is an explicit test assumption, not the previous frame-plus-battery mass subtotal.

Quantity	Test value	Unit
Complete mass m	1.2	kg
Diagonal motor spacing L	0.45	m
$J = \mathrm{diag}(J_x, J_y, J_z)$	$(0.012, 0.012, 0.022)$	kg m^2
Maximum thrust per motor $T_{\max}$	6	N
Reaction torque coefficient κ	0.015	m
Thrust response time τ_m	0.04	s
Linear drag coefficient d_v	0.12	N s/m
Gravity g	9.80665	m/s^2
Control/physics interval Δ	0.002	s
Barometer update rate	50	Hz
Gyro sample standard deviation	0.2	deg/s
Accelerometer sample standard deviation	0.005	g per axis
Barometric height standard deviation	0.05	m

The collision envelope uses $L/2 + 0.2286/2 = 0.3393$ m, rounded to 0.34 m. This includes the 9-inch propeller radius. Using $L/2$ alone would overstate obstacle clearance by approximately 11.5 cm. The actual geometry and propeller clearance must be measured; the circle is conservative in plan view.

1.2 Evidence boundaries

1. The CPU plant integrates rigid-body motion, motor lag and sensor noise. The shared library calls the same C functions as the flight firmware's math core. Host compilation is not RP2350 instruction emulation or hardware timing.

2. Two Python ROS 2 adapter nodes exchange test envelopes through Fast DDS UDP and invoke that C library. The C++ planner is compiled as a shared library. The ROS-specific C++ wrapper requires a full ROS development environment and has not been compiled here.

3. YOLO11n inference is tested independently on an official reference image. The successful route uses a known static obstacle map and simulated local odometry, not YOLO-generated obstacle ranges.

4. Isaac Sim launch attempts failed before a healthy simulation server was available: physical-device enumeration/GPU initialization failed. There is no successful Isaac physics run or Isaac recording in these results. The delivered video is an original visualization of logged CPU simulation data.

5. Physical motors remain disabled. No UART, LoRa, physical IMU, ESC, or real camera flight pipeline is exercised by the tests in this report.

Chapter 2

Linear algebra from definitions

2.1 Products, dimensions and ordering

For $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$, their product is the matrix $C \in \mathbb{R}^{m\times p}$ defined entry by entry by

$$C_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}. \tag{2.1}$$

Thus the inner dimensions must agree. Each entry is the dot product of one row of A with one column of B. For example,

$$\begin{bmatrix} 1 & \Delta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} a+\Delta c & b+\Delta d \\ c & d \end{bmatrix}, \tag{2.2}$$

$$\begin{bmatrix} a+\Delta c & b+\Delta d \\ c & d \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta & 1 \end{bmatrix} = \begin{bmatrix} a+\Delta(b+c)+\Delta^2 d & b+\Delta d \\ c+\Delta d & d \end{bmatrix}. \tag{2.3}$$

The last expression is FPF^{T}, and is the altitude covariance prediction before process noise. The order follows from the random-vector transformation; it cannot be rearranged arbitrarily.

To prove associativity rather than assume it, expand

$$[(AB)C]_{ij} = \sum_{\ell} \left(\sum_{k} A_{ik}B_{k\ell} \right) C_{\ell j} \tag{2.4}$$

$$= \sum_{k} A_{ik} \left(\sum_{\ell} B_{k\ell}C_{\ell j} \right) = [A(BC)]_{ij}. \tag{2.5}$$

Finite sums can be reordered, so the equality holds for every entry. In contrast, commutativity fails. For

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \tag{2.6}$$

$AB = \mathrm{diag}(1, 0)$ while $BA = \mathrm{diag}(0, 1)$. The transpose rule also follows entrywise: $(AB)^{\mathsf{T}}_{ij} = \sum_k A_{jk}B_{ki} = \sum_k (B^{\mathsf{T}})_{ik}(A^{\mathsf{T}})_{kj}$, hence $(AB)^{\mathsf{T}} = B^{\mathsf{T}}A^{\mathsf{T}}$.

2.2 Covariance and its transformation

Let x have mean μ and covariance $P = \mathbb{E}[(x-\mu)(x-\mu)^{\mathsf{T}}]$. For $y = Ax + b$,

$$y - \mathbb{E}[y] = A(x-\mu), \tag{2.7}$$

$$\mathrm{Cov}(y) = \mathbb{E}[A(x-\mu)(x-\mu)^{\mathsf{T}}A^{\mathsf{T}}] = APA^{\mathsf{T}}. \tag{2.8}$$

For independent zero-mean process noise w of covariance Q, $\mathrm{Cov}(Ax + w) = APA^{\mathsf{T}} + Q$: the two cross expectations vanish by independence and zero means. If the noise is correlated with the state, this last step is invalid and cross-covariance terms must be retained.

Symmetry follows because $P_{ij} = \mathbb{E}[e_i e_j] = P_{ji}$. For every v, $v^{\mathsf{T}} P v = \mathbb{E}[(v^{\mathsf{T}} e)^2] \geq 0$, so a covariance is positive semidefinite. For a symmetric 2×2 matrix, necessary and sufficient conditions are $P_{00} \geq 0$, $P_{11} \geq 0$ and $P_{00}P_{11} - P_{01}^2 \geq 0$. Covariance units vary by entry; for height and speed, P_{00} is m^2, P_{01} is m^2/s and P_{11} is $\mathrm{m}^2/\mathrm{s}^2$.

2.3 Matrix powers and exponentials

Define $A^0 = I$ and $A^{k+1} = A^k A$. For a constant square matrix,

$$e^{At} = \sum_{k=0}^{\infty} \frac{A^k t^k}{k!}. \tag{2.9}$$

The series converges absolutely: a submultiplicative norm bounds it by $\sum_k \|A\|^k |t|^k / k! = e^{\|A\||t|}$. Termwise differentiation gives $\frac{\mathrm{d}}{\mathrm{d}t} e^{At} = Ae^{At}$ and $e^{A0} = I$. Therefore $x(t) = e^{At}x(0)$ solves $\dot{x} = Ax$.

The identity $e^{A(s+t)} = e^{As}e^{At}$ holds for one constant A because its powers commute. The binomial theorem and absolutely convergent Cauchy product establish it. The stronger-looking identity $e^{A+B} = e^A e^B$ requires $AB = BA$. Expanding both sides to second order exposes the problem: the left contains $(AB + BA)/2$, while the right contains AB. Their difference is $(AB - BA)/2$ at that order.

For the altitude kinematics,

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad A^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad F = e^{A\Delta} = I + A\Delta = \begin{bmatrix} 1 & \Delta \\ 0 & 1 \end{bmatrix}. \tag{2.10}$$

Every power from A^2 onward vanishes; this truncation is exact. With $B = [0\ 1]^{\mathsf{T}}$ and a constant input acceleration over one interval,

$$x_{k+1} = Fx_k + Gu_k, \tag{2.11}$$

$$G = \int_0^{\Delta} e^{A\tau} B \, \mathrm{d}\tau = \int_0^{\Delta} \begin{bmatrix} \tau \\ 1 \end{bmatrix} \mathrm{d}\tau = \begin{bmatrix} \Delta^2/2 \\ \Delta \end{bmatrix}. \tag{2.12}$$

The integral follows from variation of constants: differentiate $e^{-At}x(t)$, integrate, then multiply by e^{At}. If $A(t)$ varies with time and does not commute at different instants, a single exponential of its integral is not generally valid.

2.4 Two distinct process-noise conventions

For one independent acceleration error η_k held constant for an entire sample, $\mathrm{Var}(\eta_k) = \sigma_a^2$ and $w_k = G\eta_k$. Thus

$$Q_{\text{held}} = \sigma_a^2 GG^{\mathsf{T}} = \sigma_a^2 \begin{bmatrix} \Delta^4/4 & \Delta^3/2 \\ \Delta^3/2 & \Delta^2 \end{bmatrix}. \tag{2.13}$$

For continuous white acceleration with spectral density q_c, however,

$$Q_{\text{continuous}} = \int_0^{\Delta} e^{A\tau} B q_c B^{\mathsf{T}} e^{A^{\mathsf{T}}\tau} \mathrm{d}\tau \tag{2.14}$$

$$= q_c \int_0^{\Delta} \begin{bmatrix} \tau^2 & \tau \\ \tau & 1 \end{bmatrix} \mathrm{d}\tau = q_c \begin{bmatrix} \Delta^3/3 & \Delta^2/2 \\ \Delta^2/2 & \Delta \end{bmatrix}. \tag{2.15}$$

These are different stochastic models with different units. The altitude C implementation uses (2.13), with $\sigma_a = 1.5$ m/s^2 per 2 ms sample. This is not a continuous spectral density. Changing the sample rate while holding that number fixed changes the effective uncertainty per second. Real noise parameters require measured bandwidth and timing.

Chapter 3

Coordinate frames and rigid-body mechanics

3.1 Frames, rotations and signs

World W is a local right-handed frame with z upward. Body B uses forward, left, up (FLU). $R = R_z(\psi)R_y(\theta)R_x(\phi)$ maps body vectors to world vectors. Here ϕ, θ, ψ are roll, pitch, yaw in radians for mechanics. The elementary matrices are

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi & c_\phi \end{bmatrix}, \quad R_y = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}, \tag{3.1}$$

$$R_z = \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{3.2}$$

where $c_\alpha = \cos\alpha$ and $s_\alpha = \sin\alpha$. Multiplication yields

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}. \tag{3.3}$$

For example, entry $(3,3)$ is $c_\theta c_\phi$, so upward thrust contributes $T c_\theta c_\phi$ to vertical force. At zero yaw and small angles, positive pitch accelerates forward and positive roll accelerates right, which is negative world y when body and world headings align. This explains the signs in the companion acceleration-to-attitude conversion.

Body angular velocity is $\omega = [p, q, r]^\mathsf{T}$. It is not generally the Euler-angle derivative:

$$\begin{bmatrix} \dot\phi \\ \dot\theta \\ \dot\psi \end{bmatrix} = \begin{bmatrix} 1 & s_\phi \tan\theta & c_\phi \tan\theta \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi / c_\theta & c_\phi / c_\theta \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}. \tag{3.4}$$

Near hover $p \simeq \dot\phi$, $q \simeq \dot\theta$, $r \simeq \dot\psi$. The independent roll/pitch filters and gain design use that approximation; the plant itself propagates a unit quaternion. Aggressive attitude changes require a coupled quaternion estimator and appropriate rate-setpoint mapping.

3.2 Newton and Euler equations

Let p_W be position, v_W velocity, and $e_3 = [0, 0, 1]^\mathsf{T}$. Then

$$\dot{p}_W = v_W, \tag{3.5}$$

$$m\dot{v}_W = Re_3 \sum_{i=1}^{4} T_i - mge_3 - d_v v_W + f_{\text{wind}}, \tag{3.6}$$

$$J\dot{\omega} = \tau - \omega \times (J\omega). \tag{3.7}$$

The rotational equation follows from angular-momentum balance. If $h_B = J\omega$, then the inertial derivative expressed in rotating body axes is $\dot{h}_B + \omega \times h_B = \tau_B$. For diagonal inertia,

$$J_x\dot{p} = \tau_x + (J_y - J_z)qr, \tag{3.8}$$

$$J_y\dot{q} = \tau_y + (J_z - J_x)rp, \tag{3.9}$$

$$J_z\dot{r} = \tau_z + (J_x - J_y)pq. \tag{3.10}$$

The inertia tensor must be positive definite and satisfy physical principal moment inequalities; the provisional diagonal values do. A measured model requires component masses and positions, using

$$J = \sum_i \left(R_i J_{i,\text{COM}} R_i^\mathsf{T} + m_i[(r_i^\mathsf{T} r_i)I - r_i r_i^\mathsf{T}] \right). \tag{3.11}$$

This is the parallel-axis theorem. It separates each component's own inertia from the extra moment caused by its distance from the vehicle centre of mass.

3.3 Motor geometry, allocation and saturation

Set $a = L/(2\sqrt{2}) = 0.159099$ m. Logical motor order is front-left (FL), front-right (FR), rear-right (RR), rear-left (RL), at $(a, a, 0), (a, -a, 0), (-a, -a, 0), (-a, a, 0)$ respectively. For $r_i = (x_i, y_i, 0)$ and thrust $f_i = (0, 0, T_i)$,

$$r_i \times f_i = \begin{bmatrix} y_i T_i \\ -x_i T_i \\ 0 \end{bmatrix}. \tag{3.12}$$

Assigning alternating reaction-torque signs $(-, +, -, +)$ gives

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ a & -a & -a & a \\ -a & -a & a & a \\ -\kappa & \kappa & -\kappa & \kappa \end{bmatrix} \begin{bmatrix} T_{FL} \\ T_{FR} \\ T_{RR} \\ T_{RL} \end{bmatrix}. \tag{3.13}$$

These are assumed spin directions; wiring must eventually be checked. In the software, u_i is a normalized *desired thrust proxy*, not a verified PWM, DShot, RPM or electrical-power relationship. Define

$$\begin{bmatrix} u_{FL} \\ u_{FR} \\ u_{RR} \\ u_{RL} \end{bmatrix} = \begin{bmatrix} 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} c \\ u_\phi \\ u_\theta \\ u_\psi \end{bmatrix}. \tag{3.14}$$

Before saturation and lag, $T_i = T_{\max} u_i$, hence

$$T = 4T_{\max} c, \quad \tau_x = 4aT_{\max} u_\phi, \quad \tau_y = 4aT_{\max} u_\theta, \quad \tau_z = 4\kappa T_{\max} u_\psi. \tag{3.15}$$

For example, positive pitch must increase rear thrust and decrease front thrust. A host regression test now checks this sign. Each output is clamped to $[0, 1]$ independently. This simple allocator does not preserve all requested torques under saturation. No tested nominal run reaches that limit.

Motor lag is represented by $\tau_m \dot{u}_i^{\text{actual}} + u_i^{\text{actual}} = u_i$. For constant commanded u_i over Δ, solve the scalar differential equation using an integrating factor:

$$u_{k+1}^{\text{actual}} = e^{-\Delta/\tau_m} u_k^{\text{actual}} + (1 - e^{-\Delta/\tau_m}) u_k. \tag{3.16}$$

This exponential update is exact for the assumed first-order actuator. Actual motors may need $T = k_T \Omega^2$, $\tau = k_Q \Omega^2$ and a measured electrical model. No battery sag or propeller aerodynamics are currently identified.

3.4 Numerical propagation

Position uses $p_{k+1} = p_k + \Delta v_k + \frac{1}{2}\Delta^2 a_k$ and velocity uses $v_{k+1} = v_k + \Delta a_k$. Angular velocity uses a first-order update of the Euler equation. Given the updated ω held over a step,

$$\delta q = \begin{bmatrix} \cos(\alpha/2) \\ \hat{\omega} \sin(\alpha/2) \end{bmatrix}, \quad \alpha = \|\omega\|\Delta, \quad q_{k+1} = q_k \otimes \delta q, \tag{3.17}$$

with the zero-speed limit handled continuously and the quaternion normalized. For $q = (s, v)$ and $r = (t, w)$, multiplication is $q \otimes r = (st - v^{\mathsf{T}} w,\ sw + tv + v \times w)$. This preserves rotational geometry for the held angular velocity, but the entire plant integrator is not an exact solution of coupled nonlinear motion. Timestep refinement and empirical identification are separate requirements.

Chapter 4

Exactly six feedback controllers

4.1 Signals and units

The six objects in the C file are roll angle, roll rate, pitch angle, pitch rate, yaw rate and altitude. A PID object may have a zero integral or derivative coefficient. Calling all six "PIDs" describes the general structure; the chosen outer loops are P and the chosen angular-rate loops are PI. No seventh vertical-rate PID is hidden in the altitude loop.

Angular gains below use **degrees** and degrees per second because that is the C interface. Physics uses radians. If an angular measurement is changed from degrees to radians, multiply corresponding numeric gains by $180/\pi$ to preserve the same normalized control output. Outer angle-to-rate P gains remain unchanged when both angle and rate units are converted together.

For a generic PID,

$$e_k = r_k - y_k, \tag{4.1}$$

$$I_k^{\text{cand}} = \text{clip}(I_{k-1} + e_k\Delta, -I_{\max}, I_{\max}), \tag{4.2}$$

$$u_k^{\text{raw}} = K_p e_k + K_i I_k^{\text{cand}} - K_d \dot{y}_k, \tag{4.3}$$

$$u_k = \text{clip}(u_k^{\text{raw}}, -u_{\max}, u_{\max}). \tag{4.4}$$

The derivative on measurement avoids setpoint derivative kick. All selected angular K_d values are zero. The altitude controller instead uses the available derivative reference v_c: $K_d(v_c - \hat{v}_z)$. The integrator candidate is accepted if unsaturated, or if the error drives an existing saturation back toward the admissible interval. Integrator and output limits are independent physical quantities.

4.2 Roll and pitch rate PI derivation

Near hover neglect cross coupling and lag for the initial design. With angular rate measured in deg/s,

$$\dot{p}_d = b_x u_\phi, \quad b_x = \frac{4aT_{\max}}{J_x}\frac{180}{\pi}, \qquad \dot{q}_d = b_y u_\theta, \quad b_y = \frac{4aT_{\max}}{J_y}\frac{180}{\pi}. \tag{4.5}$$

The plant transfer function from normalized torque command to rate is b/s. The PI controller is $C(s) = K_p + K_i/s = (K_p s + K_i)/s$. Unity feedback gives

$$1 + C(s)G(s) = 0, \tag{4.6}$$

$$1 + \frac{b(K_p s + K_i)}{s^2} = 0, \tag{4.7}$$

$$s^2 + bK_p s + bK_i = 0. \tag{4.8}$$

Choose a desired second-order polynomial $s^2 + 2\zeta\omega_n s + \omega_n^2$. Matching like powers of s proves

$$\boxed{K_p = \frac{2\zeta\omega_n}{b}, \qquad K_i = \frac{\omega_n^2}{b}, \qquad K_d = 0.} \tag{4.9}$$

For the baseline $\zeta = 0.9$, $\omega_n = 12$ rad/s. The roots have real part $-\zeta\omega_n < 0$. A state-space proof is also possible: with error $e = r - p_d$ and integral I, for constant r,

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} e \\ I \end{bmatrix} = \begin{bmatrix} -bK_p & -bK_i \\ 1 & 0 \end{bmatrix}\begin{bmatrix} e \\ I \end{bmatrix}. \tag{4.10}$$

Its determinant $\det(sI - A) = s^2 + bK_p s + bK_i$ gives the same condition.

Including the actuator lag changes the characteristic polynomial to

$$\tau_m s^3 + s^2 + bK_p s + bK_i. \tag{4.11}$$

For positive coefficients, the cubic Routh condition additionally requires $bK_p > \tau_m bK_i$, or $K_p > \tau_m K_i$. With the pole-matched gains this is $\omega_n < 2\zeta/\tau_m = 45$ rad/s. Both the baseline 12 and selected 15.6 rad/s designs meet this continuous linear condition. It is not a proof for saturation, delays, estimators or the full nonlinear aircraft.

4.3 Outer angle loops and yaw

The outer commands are

$$p_c = 3(\phi_c - \hat{\phi}), \qquad q_c = 3(\theta_c - \hat{\theta}) \quad [\text{deg/s}], \tag{4.12}$$

clamped to ± 180 deg/s. Under the ideal approximation $p_d = p_c$, $\dot{e}_\phi = -3e_\phi$, so $e_\phi(t) = e_\phi(0)e^{-3t}$. The time constant is 1/3 s and the ideal two-percent settling time is $-\ln(0.02)/3 \simeq 1.304$ s. Real cascaded dynamics are slower/different. The inner angular bandwidth is intentionally higher than this outer loop.

Yaw has *only* rate feedback in the flight core:

$$\dot{r}_d = b_\psi u_\psi, \quad b_\psi = \frac{4\kappa T_{\max}}{J_z}\frac{180}{\pi}, \quad K_{p\psi} = \frac{2(0.9)\omega_\psi}{b_\psi}, \quad K_{i\psi} = \frac{\omega_\psi^2}{b_\psi}. \tag{4.13}$$

Baseline $\omega_\psi = 4$ rad/s. There is no yaw-angle PID in the flight controller. The companion optionally creates a bounded yaw-rate request from its local-heading error; that is a navigation policy, not an extra flight PID.

4.4 One altitude PID with vertical-speed stick command

For normalized centered throttle $s_t \in [0, 1]$,

$$v_c = 4(s_t - 0.5) \quad \text{m/s}, \qquad h_{c,k+1} = h_{c,k} + v_c\Delta. \tag{4.14}$$

At centre stick, $v_c = 0$ and the target height is held. The target is initialized to the estimated height and limited to within 2 m of it to avoid unlimited reference accumulation. There is one altitude control law:

$$e_h = h_c - \hat{h}, \quad \dot{I}_h = e_h, \tag{4.15}$$

$$\delta c = K_{ph}e_h + K_{ih}I_h + K_{dh}(v_c - \hat{v}_z), \tag{4.16}$$

$$c = \mathrm{clip}\left(\frac{c_0 + \delta c}{\max(0.5, \cos\hat{\phi}\cos\hat{\theta})}, 0, 1\right). \tag{4.17}$$

The tilt factor approximately preserves vertical thrust. The factor limit prevents numerical blow-up; it is not valid recovery control at large tilt. The nominal hover command is

$$c_0 = \frac{mg}{4T_{\max}} = 0.4903325. \tag{4.18}$$

The board scaffold still has a 0.50 placeholder; SITL explicitly supplies the model-based value. A real craft requires a measured hover thrust curve.

At fixed target, near hover, let $b_h = 4T_{\max}/m = 20$ m/s^2 per command. Then $\ddot{h} = b_h \delta c$. Substitution of PID feedback gives

$$s^3 + b_h K_{dh} s^2 + b_h K_{ph} s + b_h K_{ih} = 0. \tag{4.19}$$

Choose $(s + p_h)(s^2 + 2\zeta_h \omega_h s + \omega_h^2)$, multiply explicitly:

$$s^3 + 2\zeta_h \omega_h s^2 + \omega_h^2 s + p_h s^2 + 2p_h \zeta_h \omega_h s + p_h \omega_h^2 \tag{4.20}$$

$$= s^3 + (2\zeta_h \omega_h + p_h) s^2 + (\omega_h^2 + 2p_h \zeta_h \omega_h) s + p_h \omega_h^2. \tag{4.21}$$

Coefficient matching yields

$$\boxed{\begin{aligned} K_{dh} &= (2\zeta_h \omega_h + p_h)/b_h, \\ K_{ph} &= (\omega_h^2 + 2p_h \zeta_h \omega_h)/b_h, \\ K_{ih} &= p_h \omega_h^2 / b_h. \end{aligned}} \tag{4.22}$$

Baseline $\omega_h = 1.5$, $\zeta_h = 0.9$, $p_h = 0.8$ gives $(K_p, K_i, K_d) = (0.2205, 0.09, 0.175)$. Units are respectively command/m, command/(m s), and command s/m. The integral-state bound is 2 m s and the altitude-correction bound is 0.35 command. Actuator lag increases the order:

$$\tau_m s^4 + s^3 + b_h K_d s^2 + b_h K_p s + b_h K_i. \tag{4.23}$$

For positive coefficients the quartic Routh conditions reduce to

$$b_h K_d > \tau_m b_h K_p, \qquad (b_h K_d)(b_h K_p) > \tau_m (b_h K_p)^2 + b_h K_i. \tag{4.24}$$

These hold for the baseline and selected simulation gains. Discrete feedback and filtering still require the actual closed-loop tests.

4.5 A reproducible gain family, not hidden hand tuning

Three candidates use scale $\lambda \in \{0.7, 1, 1.3\}$ with $\omega_n = 12\lambda$, $\omega_\psi = 4\lambda$, $\omega_h = 1.5\lambda$, $p_h = 0.8\lambda$; outer P stays 3. Thus rate K_p scales as λ, rate K_i as λ^2, altitude K_p as λ^2, altitude K_i as λ^3, and altitude K_d as λ. Full numeric tables and the declared selection score appear in the results chapter. The selected family is the best of these three under that score, not a globally optimal tune.

Chapter 5

Kalman filtering: proof and implemented equations

5.1 Derivation of the gain

Let $\hat{x}^-$ be the prior estimate, $e^- = x - \hat{x}^-$, with covariance P^-. A measurement is $z = Hx + v$ with independent noise covariance R. The innovation is $\nu = z - H\hat{x}^- = He^- + v$. Hence

$$S = \mathrm{Cov}(\nu) = HP^-H^\mathsf{T} + R. \tag{5.1}$$

Use $\hat{x}^+ = \hat{x}^- + K\nu$. Then

$$e^+ = (I - KH)e^- - Kv. \tag{5.2}$$

Taking its covariance, retaining both independent-noise contributions, gives the Joseph form

$$P^+ = (I - KH)P^-(I - KH)^\mathsf{T} + KRK^\mathsf{T}. \tag{5.3}$$

Expanding this expression gives $P^+ = P^- - KHP^- - P^-H^\mathsf{T}K^\mathsf{T} + KSK^\mathsf{T}$. The expected squared error is $\mathbb{E}[e^{+\mathsf{T}}e^+] = \mathrm{tr}(P^+)$. Complete the square using $K_* = P^-H^\mathsf{T}S^{-1}$:

$$P^+ = P^- - P^-H^\mathsf{T}S^{-1}HP^- + (K - K_*)S(K - K_*)^\mathsf{T}. \tag{5.4}$$

The last term is positive semidefinite when S is positive definite. Its trace is minimized by $K = K_*$, proving the Kalman gain. Gaussianity is not required for this best linear update; Gaussian priors/noises additionally make it the conditional-mean Bayesian update and justify normal intervals.

5.2 Scalar one-angle filter

A strictly scalar attitude model (calibrated gyro bias assumed known) is

$$\hat{\theta}_k^- = \hat{\theta}_{k-1}^+ + \Delta(\omega_{g,k} - b), \tag{5.5}$$

$$P_k^- = P_{k-1}^+ + \Delta^2\sigma_g^2, \tag{5.6}$$

$$K_k = \frac{P_k^-}{P_k^- + R_\theta}, \tag{5.7}$$

$$\hat{\theta}_k^+ = \hat{\theta}_k^- + K_k(\theta_{a,k} - \hat{\theta}_k^-), \tag{5.8}$$

$$P_k^+ = (1 - K_k)^2P_k^- + K_k^2R_\theta. \tag{5.9}$$

This is the literal one-state interpretation of "1D Kalman". K is not an arbitrary fixed mixing percentage: it changes with prior uncertainty, sample interval and measurement noise.

5.3 Actual MPU filter: one axis, two states

The C implementation estimates *one angle at a time*, independently for roll and pitch, but also estimates gyro bias. Its state dimension is therefore two, even though it is often informally called a "1D attitude filter". This document names that distinction explicitly.

$$x = \begin{bmatrix}\theta \\ b\end{bmatrix}, \quad F = \begin{bmatrix}1 & -\Delta \\ 0 & 1\end{bmatrix}, \quad B = \begin{bmatrix}\Delta \\ 0\end{bmatrix}, \quad H = \begin{bmatrix}1 & 0\end{bmatrix}. \tag{5.10}$$

The bias is subtracted from the gyro, so the upper-right entry is negative. With independent per-sample gyro variance σ_g^2 and continuous bias random-walk spectral density q_b,

$$Q = \begin{bmatrix}\sigma_g^2\Delta^2 + q_b\Delta^3/3 & -q_b\Delta^2/2 \\ -q_b\Delta^2/2 & q_b\Delta\end{bmatrix}. \tag{5.11}$$

The bias contribution follows by integrating $q_b[-\tau\ 1]^\mathsf{T}[-\tau\ 1]$ from zero to Δ. Expanding FPF^T entry by entry gives

$$P_{00}^- = P_{00} - \Delta(P_{01} + P_{10}) + \Delta^2 P_{11} + \sigma_g^2\Delta^2 + q_b\Delta^3/3, \tag{5.12}$$

$$P_{01}^- = P_{01} - \Delta P_{11} - q_b\Delta^2/2, \tag{5.13}$$

$$P_{10}^- = P_{10} - \Delta P_{11} - q_b\Delta^2/2, \tag{5.14}$$

$$P_{11}^- = P_{11} + q_b\Delta. \tag{5.15}$$

Since $H = [1, 0]$, the matrix inverse in the gain formula is just a scalar division:

$$S = P_{00}^- + R_\theta, \quad K = \frac{1}{S}\begin{bmatrix}P_{00}^- \\ P_{10}^-\end{bmatrix}, \quad \begin{bmatrix}\hat\theta^+ \\ \hat b^+\end{bmatrix} = \begin{bmatrix}\hat\theta^- \\ \hat b^-\end{bmatrix} + \begin{bmatrix}K_\theta \\ K_b\end{bmatrix}(\theta_a - \hat\theta^-). \tag{5.16}$$

K_θ is dimensionless; K_b has units s^{-1} and is normally negative with this sign convention. A positive angle innovation should reduce an estimated positive gyro bias that has been over-subtracted.

The initialization is $P_0 = \mathrm{diag}(1, 0.01)$, $\sigma_g^2 = 0.04$, $q_b = 0.0001$ and $R_\theta = 4$ in degree-based units. At $\Delta = 0.002$ s, the first prediction yields

$$P_{00}^- = 1.000000200000267, \tag{5.17}$$

$$P_{01}^- = P_{10}^- = -0.0000200002, \tag{5.18}$$

$$P_{11}^- = 0.0100002, \tag{5.19}$$

$$S = 5.000000200000267, \tag{5.20}$$

$$K_\theta \simeq 0.200000032, \qquad K_b \simeq -0.00000400004\ \mathrm{s}^{-1}. \tag{5.21}$$

The numerical first gain is not the eventual gain; plots and end-of-run values are generated from the actual C trace.

5.4 Accelerometer observations and their limitation

For calibrated specific force f_B in units of g, the small-acceleration gravity observations are

$$\phi_a = \mathrm{atan2}(f_y, f_z), \qquad \theta_a = \mathrm{atan2}(-f_x, \sqrt{f_y^2 + f_z^2}). \tag{5.22}$$

Convert these to degrees before feeding the C filter. The accelerometer measures specific force, not gravity alone: $f_B = R^\mathsf{T}(a_W + ge_3)$ in SI units. During sustained translation, these angle observations are biased. Fixed $R_\theta = 4\ \mathrm{deg}^2$ is a provisional robustness choice, not the variance implied directly by 0.005 g sensor noise. Their nonlinear Jacobian would propagate raw measurement covariance; motion adds further model error. This explains why a mathematically valid covariance recursion alone does not prove that the reported confidence intervals are calibrated in real flight.

5.5 Altitude filter: state, measurements and expanded products

Use $x = [h, v_z]^\mathsf{T}$ and the exact F, G derived earlier. Corrected world-up acceleration is the control input. The C state prediction is

$$\hat{h}^- = \hat{h} + \Delta\hat{v}_z + \tfrac{1}{2}\Delta^2 a_z, \qquad \hat{v}_z^- = \hat{v}_z + \Delta a_z. \tag{5.23}$$

The covariance equations from $FPF^\mathsf{T} + Q_{\text{held}}$ are

$$P_{00}^- = P_{00} + \Delta(P_{01} + P_{10}) + \Delta^2 P_{11} + \tfrac{1}{4}\sigma_a^2\Delta^4, \tag{5.24}$$

$$P_{01}^- = P_{01} + \Delta P_{11} + \tfrac{1}{2}\sigma_a^2\Delta^3, \tag{5.25}$$

$$P_{10}^- = P_{10} + \Delta P_{11} + \tfrac{1}{2}\sigma_a^2\Delta^3, \tag{5.26}$$

$$P_{11}^- = P_{11} + \sigma_a^2\Delta^2. \tag{5.27}$$

Barometric height has $z_h = Hx + v_h$, $H = [1, 0]$, $R_h = 0.0025$ m^2 in SITL. The gains are

$$K_h = \frac{P_{00}^-}{P_{00}^- + R_h}, \qquad K_v = \frac{P_{10}^-}{P_{00}^- + R_h}. \tag{5.28}$$

Here K_v is positive: a higher observed altitude tends to raise both estimated height and upward speed. A barometer reading is corrected *once when fresh*, every ten control steps. Reusing the same reading as if it were ten independent measurements would understate uncertainty. The gains logged between observations are the most recent gains, not ten new corrections.

SITL starts at $P_0 = \text{diag}(0.04, 0.1)$, with $\sigma_a = 1.5$. The first 2 ms prediction gives

$$P^- = \begin{bmatrix} 0.040000400009 & 0.000200009 \\ 0.000200009 & 0.100009 \end{bmatrix}, \tag{5.29}$$

$$S = 0.042500400009, \tag{5.30}$$

$$K_h \simeq 0.941177024, \qquad K_v \simeq 0.00470605 \text{ s}^{-1}. \tag{5.31}$$

The firmware scaffold's boot defaults are instead $P_0 = \text{diag}(1, 1)$ and $R_h = 2$ m^2. The host simulation deliberately supplies the documented test configuration through its reset interface. Neither is a measured BMP280 noise model. They must not be confused when reproducing results.

5.6 Joseph multiplication in full detail

Let a symmetric prior be $P^- = \begin{bmatrix} a & b \\ b & d \end{bmatrix}$, measurement variance r, and gain $K = [k_0, k_1]^\mathsf{T}$. For $H = [1, 0]$,

$$M = I - KH = \begin{bmatrix} 1 - k_0 & 0 \\ -k_1 & 1 \end{bmatrix}. \tag{5.32}$$

Writing $z = 1 - k_0$ and multiplying row by column,

$$MP^- = \begin{bmatrix} za & zb \\ b - k_1 a & d - k_1 b \end{bmatrix}, \tag{5.33}$$

$$MP^-M^\mathsf{T} = \begin{bmatrix} z^2 a & z(b - k_1 a) \\ z(b - k_1 a) & d - 2k_1 b + k_1^2 a \end{bmatrix}, \tag{5.34}$$

$$KrK^\mathsf{T} = r \begin{bmatrix} k_0^2 & k_0 k_1 \\ k_0 k_1 & k_1^2 \end{bmatrix}. \tag{5.35}$$

Adding gives exactly the four expressions implemented in `covariance_correct`. The routine averages the two off-diagonal prior entries to remove small numerical asymmetry, and writes the same posterior off-diagonal value twice. Joseph form is a sum of positive semidefinite matrices in exact arithmetic. Finite-precision roundoff still requires monitoring; a failed covariance validity check must not be disguised by claiming the physical filter is healthy.

5.7 Gravity compensation, lidar and observability

The upward acceleration supplied to the filter is

$$a_z = -\sin\hat{\theta}\, f_x + \sin\hat{\phi}\cos\hat{\theta}\, f_y + \cos\hat{\phi}\cos\hat{\theta}\, f_z - g, \tag{5.36}$$

with f in m/s^2. At rest and level it is zero. In the CPU model this uses the preceding C attitude estimate, not the exact plant attitude, so attitude error affects height estimation. The resulting correlated errors are not fully represented by this two-state covariance model.

For flat ground and a truly downward aligned rangefinder, approximate $h = d\cos\phi\cos\theta$ after accounting for the sensor lever arm. Its variance can be approximated by $R_h = J_d \Sigma J_d^{\mathsf{T}}$, with

$$J_d = \begin{bmatrix} \cos\phi\cos\theta & -d\sin\phi\cos\theta & -d\cos\phi\sin\theta \end{bmatrix}. \tag{5.37}$$

Terrain slope, missing returns, tilt and range limits violate this simple measurement model. A downward TOFSense-F2 Mini is therefore a useful future height aid, but it is not fused in the current trace. A front sensor sees only one ray; it cannot establish a full safe corridor by itself.

For the altitude pair, the two-observation observability matrix is

$$\mathcal{O} = \begin{bmatrix} H \\ HF \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & \Delta \end{bmatrix}, \quad \det\mathcal{O} = \Delta \neq 0. \tag{5.38}$$

Thus repeated independent height measurements make both height and speed observable. The analogous angle-bias determinant is $-\Delta$. With all height measurements missing, integration alone drifts and no longer bounds height uncertainty. Loss of sensor aiding requires a defined, tested degraded mode; generic zero motor output in flight is not a safe landing policy.

Chapter 6

Uncertainty, statistical checks and error accounting

6.1 What standard deviation does and does not mean

For a scalar zero-mean Gaussian estimation error $e \sim \mathcal{N}(0, P)$, $\sigma = \sqrt{P}$. The standardized variable $u = e/\sigma$ has unit normal density $\exp(-u^2/2)/\sqrt{2\pi}$. Therefore

$$\Pr(|e| \le c\sigma) = \int_{-c}^{c} \frac{e^{-u^2/2}}{\sqrt{2\pi}} \mathrm{d}u, \tag{6.1}$$

which is approximately 68.27% for $c = 1$ and 95.45% for $c = 2$. A covariance is a model prediction, not a measured error bound. Gaussian intervals can fail under bias, bad calibration, outliers, latency or incorrect process-noise assumptions.

For N Monte Carlo errors e_i at a fixed time, compute

$$\bar{e} = \frac{1}{N}\sum_i e_i, \quad s_e = \sqrt{\frac{\sum_i (e_i - \bar{e})^2}{N-1}}, \quad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_i e_i^2}. \tag{6.2}$$

The $N - 1$ denominator makes sample variance unbiased for independent samples. RMSE includes bias; standard deviation describes spread about the sample mean. For $N = 500$ and true coverage 0.9545, the binomial standard error of an observed two-sigma coverage fraction is $\sqrt{0.9545(1 - 0.9545)/500} \simeq 0.00932$ at one independent time point. Successive time points within a trajectory are correlated, so pooling them does not create that many independent trials.

6.2 Innovation consistency and physical-parameter errors

For a scalar observation, normalized innovation squared is $\mathrm{NIS} = \nu^2/S$. Under the correctly specified linear Gaussian model, its mean is one and its distribution is χ^2_1. Persistent large values suggest underestimated uncertainty, bias, bad timing or sensor failures; very small values can indicate overestimated noise. This is a diagnostic, not an automatic proof of sensor validity. The C trace records angle innovation and its variance. Altitude innovation variance can be reconstructed on fresh corrections from $S = R_h/(1 - K_h)$, subject to roundoff and $K_h < 1$.

For a physical parameter vector η with covariance Σ_η, a first-order output uncertainty is

$$\Sigma_f \simeq J_f \Sigma_\eta J_f^{\mathsf{T}}, \qquad (J_f)_{ij} = \frac{\partial f_i}{\partial \eta_j}. \tag{6.3}$$

For example, $c_0 = mg/(4T_{\max})$ has derivatives

$$\frac{\partial c_0}{\partial m} = \frac{g}{4T_{\max}}, \qquad \frac{\partial c_0}{\partial T_{\max}} = -\frac{mg}{4T_{\max}^2}. \tag{6.4}$$

With independent relative errors this gives

$$\left(\frac{\sigma_{c_0}}{c_0}\right)^2 \simeq \left(\frac{\sigma_m}{m}\right)^2 + \left(\frac{\sigma_T}{T_{\max}}\right)^2. \tag{6.5}$$

For $b_x \propto aT_{\max}/J_x$, the logarithmic first-order perturbation is $\delta b_x/b_x \simeq \delta a/a + \delta T/T - \delta J_x/J_x$. Correlations must be included if, for example, moving a battery changes both centre of mass and inertia. The current mass/thrust perturbations are scenario tests, not measured probability distributions for the actual aircraft.

6.3 Separate error sources

The report distinguishes: floating-point and timestep error; noisy sensors; calibration error; structural model error; uncertain mass/inertia/thrust; latency and stale-data error; and perception/localization error. Running the same approximate equations very precisely only reduces numerical error. It does not eliminate structural or identification error. Six successful scenarios provide evidence for those scenarios, not a probability of safe flight in arbitrary environments.

Chapter 7

Companion navigation and perception

7.1 A bounded static-map baseline

The common C++ planner discretizes $x \in [-2, 10]$, $y \in [-4, 4]$ into 0.2 m cells. A circle obstacle of radius r_i blocks centres within $r_i + 0.65$ m. The 0.65 m inflation includes the 0.34 m propeller envelope and additional tracking/grid allowance. Eight-connected A* uses edge length 0.2 or $0.2\sqrt{2}$ and Euclidean heuristic. Diagonal motion through blocked corners is rejected. Since Euclidean straight-line distance never exceeds path length, the heuristic is admissible. It does not prove continuous-space safety under arbitrary tracking errors; actual trajectory clearance is computed separately.

For waypoint error e_{xy}, set desired horizontal velocity $v_c = \mathrm{clip}_{\|\cdot\| \le 0.8}(e_{xy})$ and acceleration $a_{xy,c} = \mathrm{clip}_{\mathrm{component},1.4}[1.8(v_c - v_{xy})]$. With yaw ψ, transform world horizontal acceleration into body heading:

$$\begin{bmatrix} a_{xB} \\ a_{yB} \end{bmatrix} = \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} a_{xW} \\ a_{yW} \end{bmatrix}. \tag{7.1}$$

The approximate requests are $\theta_c = \arctan(a_{xB}/g)$ and $\phi_c = -\arctan(a_{yB}/g)$, limited to 15 degrees. The waypoint advances within 0.4 m. Goal acceptance requires horizontal error below 0.2 m, vertical error below 0.2 m and horizontal speed below 0.15 m/s, followed by one second dwell in the route test. Flights begin already airborne at 1.5 m; these tests do not include takeoff, landing or recovery from a crash.

7.2 ROS 2 data contract

The ROS C++ source uses a custom timestamped AutonomySetpoint message with roll angle, pitch angle, yaw rate, centered throttle, validity and goal status. Angles are radians at the ROS boundary and degrees inside the C core. A 250 ms odometry freshness gate invalidates companion output. Local pose and velocity must use one consistent frame; the wrapper requires a documented adapter that transforms standard body-frame Odometry twist to world velocity. Ordinary body twist must not be silently interpreted as world velocity. No setpoint arms hardware. A future physical bridge must enforce pilot mode authority, timestamp/sequence checks, watchdogs and a tested degraded mode.

The executed DDS test uses two Python adapter nodes with numeric test envelopes, carrying sensor samples, commands, motor proxies and estimator diagnostics. It exercises actual DDS serialization/transport and C execution, but does not validate generated custom-message C++ bindings, physical serial framing or a complete real-time companion system.

7.3 Why YOLO is not a range sensor

For a calibrated pinhole camera, optical coordinates satisfy $u = f_x X/Z + c_x$ and $v = f_y Y/Z + c_y$. If a registered depth image supplies Z,

$$X = (u - c_x)Z/f_x, \qquad Y = (v - c_y)Z/f_y. \tag{7.2}$$

Without depth or another geometric constraint, multiplying (X, Y, Z) by any positive scale leaves (u, v) unchanged. This proves the metric-scale ambiguity of a single detection image. A bounding box cannot supply reliable distance by itself. The tested geometry utility rejects invalid range values and back-projects *all* valid sampled depth pixels, including objects outside YOLO's recognized classes. Bounding-box median depth is only a semantic annotation.

Transform each point with calibrated extrinsics and synchronized pose: $p_W = R_{WB}(R_{BC}p_C + t_{BC}) + t_{WB}$. Future map insertion needs ground filtering, voxelization, obstacle growth, clearing/decay, temporal consistency and replanning. None of those steps can be replaced by the standalone bus-image detection test. The same ROS C++ navigation core can run on Pi or Jetson; perception acceleration and camera drivers remain platform-specific concerns.

Chapter 8

Reproduction and next physical measurements

The evidence is dated 20 October 2026. Run the host C checks, build SITL libraries, execute the route suite, execute the ROS DDS adapter test, and render graphs/video using the scripts listed in the results chapter. All stochastic seeds and assumed parameters are saved alongside the traces. The video plays one simulated second per second; producing it offline is not proof of live 500 Hz execution. DDS wall-clock percentiles include Windows scheduling.

To make the model physically precise, measure complete takeoff mass and centre of mass; identify motor/propeller thrust and current versus command at several battery voltages; estimate actuator lag; measure inertia or assemble component inertias; measure vibration, gyro Allan behaviour, accelerometer and barometer noise after mounting; measure communication age/jitter; calibrate camera intrinsics/extrinsics and rangefinder offsets. Until then all numeric PID values in this report are simulation candidates. The LiPo's capacity and C-rating alone do not establish usable energy, voltage sag, thrust reserve or safe cell condition.

Primary implementation references for later integration are NVIDIA's Isaac Sim requirements and Ultralytics' prediction and model export documentation. The algebra and gain derivations above are developed explicitly for this project.

Chapter 9

Recorded numerical results

9.1 Six PID gain tables

These values are read from the saved simulation summary. Order is roll angle, roll rate, pitch angle, pitch rate, yaw rate, altitude. All angular interfaces here are degree-based. The boot C defaults are the $\lambda = 1$ table; the selected video explicitly loads the $\lambda = 1.3$ table through the host reset API. Neither table is authorized for physical flight.

9.1.1 Baseline

Controller	K_p	K_i	K_d
Roll angle	3	0	0
Roll rate	0.00118476874	0.00789845828	0
Pitch angle	3	0	0
Pitch rate	0.00118476874	0.00789845828	0
Yaw rate	0.00767944893	0.0170654412	0
Altitude	0.220500007	0.0900000036	0.174999997

9.1.2 Selected simulation

Controller	K_p	K_i	K_d
Roll angle	3	0	0
Roll rate	0.00154019939	0.013348395	0
Pitch angle	3	0	0
Pitch rate	0.00154019939	0.013348395	0
Yaw rate	0.00998328347	0.0288405959	0
Altitude	0.372644991	0.197730005	0.227500007

9.2 Selection and route suite

Every candidate is run on the same nominal seed 2350. A successful candidate must avoid collision, end within 0.3 m of the 3D goal, and satisfy the navigator's arrival conditions followed by one second dwell. Among successful nominal candidates, the declared score is

$$J = t_f + 20\,\mathrm{RMSE}_h + 10\,\|p_f - p_g\| + \frac{2}{d_{\min} + 0.01}. \tag{9.1}$$

The coefficients are preference weights with implicit unit conversion; this is not a dimensionless universal cost or a statistical estimator. Ranking depends on clearance definition and weights. After including the complete propeller envelope, the selected nominal case is nominal-1.3.

Case	t_f [s]	Height RMS [m]	Final error [m]	Clearance [m]	Success
nominal-0.7	21.45	0.01057	0.13837	0.29061	yes
nominal-1.0	21.50	0.01115	0.14035	0.29897	yes
nominal-1.3	21.50	0.01199	0.14165	0.30058	yes
heldout-heavy	21.55	0.02968	0.14497	0.31306	yes
heldout-weak-thrust	21.60	0.03310	0.14106	0.29882	yes
heldout-light	20.35	0.03887	0.03652	0.27065	yes

Held-out scenarios do not change the selected gains: heavy mass is $1.15m$ (seed 2351), weak thrust is $0.85T_{\max}$ (seed 2352), and light mass/strong thrust is $0.85m$, $1.1T_{\max}$ (seed 2353). The controller hover feed-forward is kept nominal, so the altitude integrator must compensate. These three perturbations are not an exhaustive robustness region. Wind is a one-second horizontal force of $(0.4, -0.3, 0)$ N between 7 and 8 seconds. All saved scenarios have zero motor-command saturation fraction. Initial motor thrust is balanced for the physical test mass because these are airborne route tests, not spool-up tests.

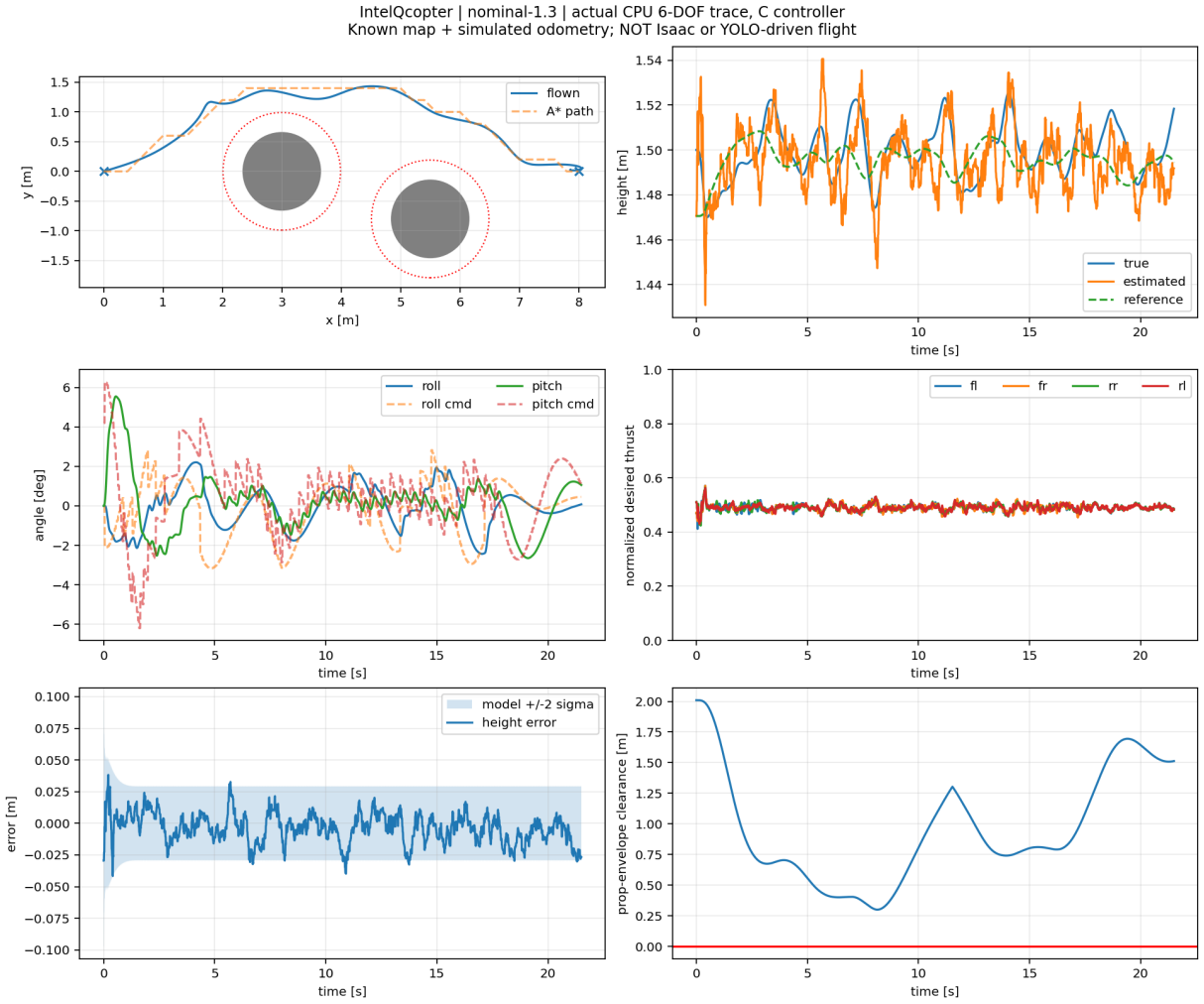


Figure 9.1: Original plots of saved CPU trace. Dashed circles include the propeller envelope; the A* planner uses a larger planning inflation.

9.3 Kalman gains, standard deviation and distributions

The final logged gains in the selected route are

$$K_\theta = 0.00096639531, \qquad K_b = -0.000223559371 \text{ s}^{-1}, \tag{9.2}$$

$$K_h = 0.0834309161, \qquad K_v = 0.181649998 \text{ s}^{-1}. \tag{9.3}$$

These are end-of-run time-varying values, not preselected fixed constants. The selected route's roll and pitch estimation RMSEs are 0.75897 and 1.29857 degrees. Height error lies within its predicted two-sigma interval at 98.419% of logged times. This is one correlated nonlinear trajectory and must not be interpreted as 500 independent statistical trials.

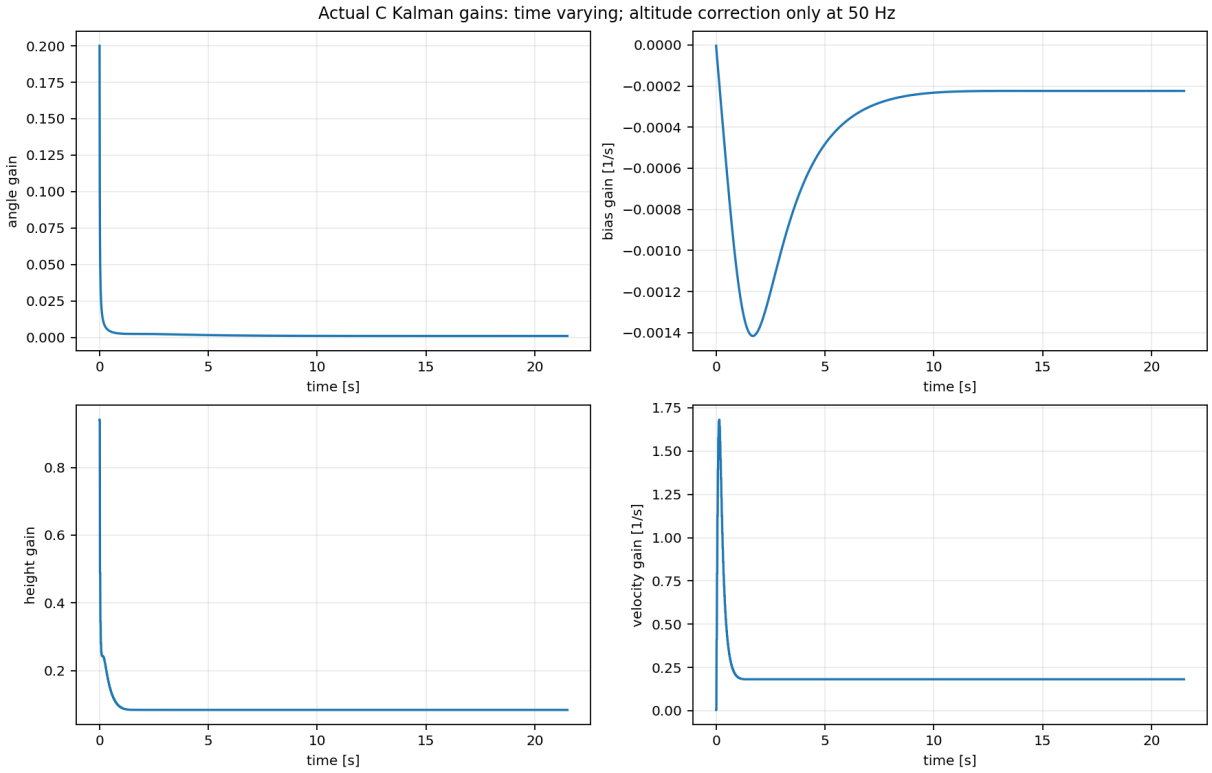


Figure 9.2: Actual gain evolution. Altitude gains are held between fresh barometer observations.

9.3.1 Independent Monte Carlo experiments

Both experiments call the production C estimator functions, with 500 synthetic independent trials of 2500 steps (5 seconds each). The angle test uses seed 2350, prior angle sigma 1 degree, bias sigma 0.1 deg/s, gyro sample sigma 0.2 deg/s, accelerometer-angle sigma 2 degrees, and bias random-walk sigma 0.01 deg/s/$\sqrt{\text{s}}$. It includes a prescribed eight-degree ramp. The altitude test uses seed 2352, height prior sigma 0.2 m, speed prior sigma $\sqrt{0.1}$ m/s, unobserved independent acceleration errors of sigma 1.5 m/s^2 per sample, and height observations of sigma 0.05 m at 50 Hz. The altitude truth is propagated by exactly the held-acceleration stochastic model used to derive Q. This tests statistical consistency under a matched model, not realistic unknown airframe physics.

At final logged time	Mean error	Empirical std	Predicted std	2σ coverage
Angle [deg]	-0.004526	0.078910	0.079017	95.00%
Height [m]	-0.000296	0.013305	0.014442	96.80%

The angle generator samples bias Brownian motion at each endpoint while the covariance includes the continuous interval integral; the difference is small at 2 ms, but the test is not an exact continuous-noise sampler. Normalized innovation checks and real sensor data are still needed before using these intervals as engineering confidence claims.

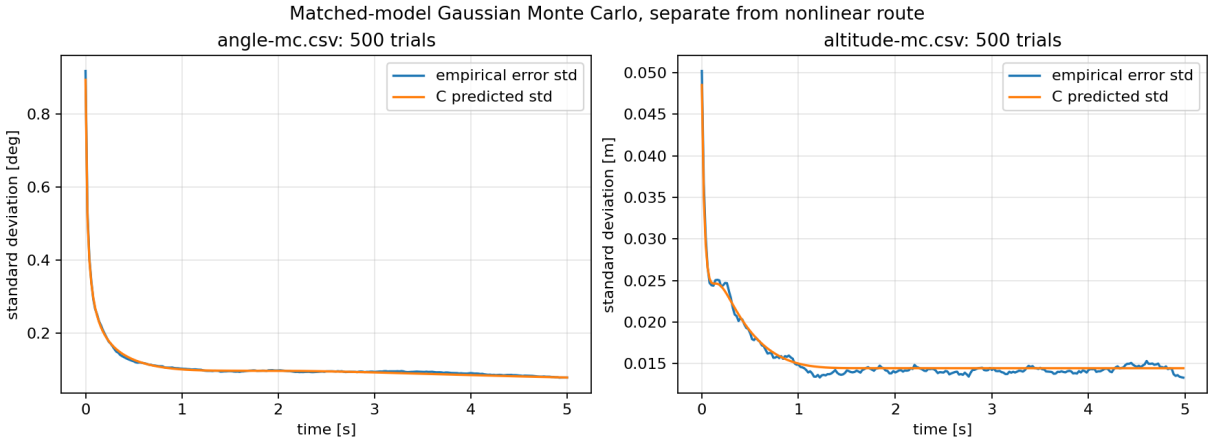


Figure 9.3: Empirical spread versus model-predicted spread; original C Monte Carlo data.

9.4 Numerical refinement and algebra checks

Host tests independently expand FPF^{T} and the Joseph correction, check covariance symmetry and nonnegative determinants, and check positive pitch increases rear thrust in FLU coordinates. Separate tests check rotation orthogonality/determinant, matrix noncommutativity, nilpotence, stale navigation, NaN navigation, out-of-map goals, occupied goals, blocked maps, depth-plane back-projection and rejection of invalid depth. Linearized angular-rate and altitude polynomials including the motor lag have strictly left-half-plane roots for both baseline and selected gains. This does not include outer-loop, estimator or sampled-data stability in that root test.

The selected case is repeated with unchanged 500 Hz sensing and control but physical integration at 1 ms and 0.5 ms. The maximum aligned-time position differences from the original 2 ms integration are:

Physics interval	Max difference [m]	Route success
1.0 ms	0.0017292	yes
0.5 ms	0.0025954	yes

The comparison bounds disagreement between these tested discretizations only. It is not a rigorous global integration-error bound and does not measure physical model error. Noise samples remain identical at the control interval.

9.5 Executed ROS 2 communication test

Two same-process Python adapter nodes exchanged 10,751 strictly sequenced request/response frames through Fast DDS UDPv4. Every received sensor/command envelope invoked the production C core; the result returned through DDS to the CPU plant. The best gain scale was used and the route succeeded. The median wall-clock round trip was 0.4693 ms, the 99th percentile 0.7514 ms, and the maximum 14.0538 ms. The maximum can exceed the 2 ms control period; this is not a hard-real-time 500 Hz scheduling demonstration. The RP2350 is intended to own that local loop; slow companion work must not block it. The two-node test is not an independent-process or wireless benchmark and does not include a real UART. The complete C++ ROS wrappers have been authored but not built on this PC.

9.6 Executed YOLO and USD tests

YOLO11n performed actual GPU inference on the official bus reference image: 5 detections. The warm measured inference phase reported 9.850 ms and the end-to-end warm call took 24.618 ms on this PC. One reference image is neither a perception benchmark nor a navigation safety test. Model and input SHA-256 hashes are saved in the JSON report. The separately executed OpenCV DNN CPU test loaded the exported ONNX model, applied the C++ node's square-letterbox/raw-output/NMS contract, and produced 5 detections in an inference call of 97.894 ms. This exercised the portable inference engine through Python, not a running ROS C++ node. The raw output shape was $1 \times 84 \times 8400$ and the model hash is saved. The ROS C++ YOLO source supports a raw YOLO11 ONNX model, letterboxing, confidence thresholding, NMS and synchronized registered-depth annotation. That C++ wrapper has not been compiled or executed here. The separate portable USD scene is authored with positive assumed mass/inertia, four 9047 rotor visuals, a conservative collision envelope, ground and obstacles. Its parser and attributes are validated separately; it is not an executed Isaac scene.

9.7 Files and reproducibility

Project paths below are relative to the repository root, not shell commands that install system software.

C firmware `flight_controller/flight_controller.c`.

Host checks `tools/run_flight_controller_tests.ps1`.

Host libraries `tools/build_sitl.ps1`.

CPU route `sim/sitl/run.py`.

Algebra/refinement `sim/sitl/analysis_tests.py`.

DDS integration `sim/ros2_integration_test.py` with bundled Isaac Python 3.12.

Perception reference `sim/yolo_reference_test.py`.

USD authoring `sim/isaac/build_stage.py`.

Graphs/video `sim/sitl/visualize.py`.

Data `artifacts/simulation/2026-10-20/summary.json`, per-case CSVs, Monte Carlo CSVs, `analysis-tests.json`, DDS and perception subdirectories.

Original logs `artifacts/test-runs/2026-10-20/`.

Video `artifacts/simulation/2026-10-20/best-flight-cpu.mp4`.

The video visualizes the selected saved trajectory at real-time playback speed with live-progress graphs. It is not an Isaac recording, physical flight, or a demonstration of YOLO-driven obstacle avoidance. Isaac remains blocked by the recorded GPU startup failure on this workstation.